



Daffodil
International
University



LAB PART PROJECT

Course Code : CIS 311L

Course Title : Network Security and Cryptography

Submitted by

Md. Rayhan Parvaz

ID : 242-16-007

Section : 20_A

Submitted to

Sifat Jahan Shorna

Lecturer,

Department of CIS, DIU

Date of Submission : 7 August 2026

Task 1(a): AES-128 Encryption & Decryption (JavaScript)

Source Code:

```
const crypto = require('crypto');
const readline = require('readline');

const algorithm = 'aes-128-cbc';
const iv = crypto.randomBytes(16);

function aesEncrypt(text, key) {
    const cipher = crypto.createCipheriv(algorithm, key, iv);

    let encrypted = cipher.update(text, 'utf8', 'hex');
    encrypted += cipher.final('hex');

    return encrypted;
}

function aesDecrypt(encryptedText, key) {
    const decipher = crypto.createDecipheriv(algorithm, key, iv);

    let decrypted = decipher.update(encryptedText, 'hex', 'utf8');
    decrypted += decipher.final('utf8');

    return decrypted;
}

const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout
});

rl.question("Enter Plain Text: ", function(text) {

    console.log("\n===== AES-128 Encryption & Decryption =====\n");
    console.log("Plain Text:", text);

    let pass = "MySecretKey123";
    const key = crypto.scryptSync(pass, "hello", 16);

    console.log("\nGenerated Key:");
    console.log(key);
```

```

    console.log("\nKey Size:");
    console.log(key.length + " bytes (128-bit)");

    const encryptedText = aesEncrypt(text, key);

    console.log("\nEncrypted Text:");
    console.log(encryptedText);

    const decryptedText = aesDecrypt(encryptedText, key);

    console.log("\nDecrypted Text:");
    console.log(decryptedText);

    console.log("\n=====");

    rl.close();
  });
});

```

OUTPUT :

```

● PS C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab> node aes128.js
Enter Plain Text: Rayhan Parvaz

===== AES-128 Encryption & Decryption =====

Plain Text: Rayhan Parvaz

Generated Key:
<Buffer e6 dc 83 ea a4 bd 06 79 c9 ac 75 ea 23 79 a3 69>

Key Size:
16 bytes (128-bit)

Encrypted Text:
53a79c57113924eccde9dbfaf0df0d88

Decrypted Text:
Rayhan Parvaz

=====
○ PS C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab>

```

Task 1(b): SHA-256 Hashing (Python)

Source Code :

```
import hashlib

def sha256(message):

    hash1 = hashlib.sha256()
    hash1.update(message.encode())

    return hash1.hexdigest()

def sha256_verify(message):

    hash2 = hashlib.sha256()
    hash2.update(message.encode())

    return hash2.hexdigest()

message = input("Enter Plain Text: ")

hashed_value1 = sha256(message)
hashed_value2 = sha256_verify(message)

print("\n===== SHA-256 Hash Verification =====\n")

print("Plain Text:", message)

print("\nHash Value 1:")
print(hashed_value1)
print("Length:", len(hashed_value1))

print("\nHash Value 2:")
print(hashed_value2)
print("Length:", len(hashed_value2))

if hashed_value1 == hashed_value2:
    print("\nVerification Result: Hashes Match")
```

```
else:
    print("\nVerification Result: Hashes Don't Match")

if len(message) > 0:
    modified_message = message[:-1] + "X"
else:
    modified_message = "X"

modified_hash = sha256(modified_message)

print("\nModified Message:")
print(modified_message)

print("\nModified Hash:")
print(modified_hash)
print("Length:", len(modified_hash))

print("\nAvalanche Effect:")

if hashed_value1 != modified_hash:
    print("Changing one character completely changes the SHA-256 hash.")
else:
    print("Hashes are the same.")

print("\n=====")
```

Output:

```
python.exe c:/Users/hp/OneDrive/Documents/Desktop/NSC_Lab/sha256.py
Enter Plain Text: Rayhan Parvaz

===== SHA-256 Hash Verification =====

Plain Text: Rayhan Parvaz

Hash Value 1:
a0badb200487fc4ce88b991c859ca928391ad8ed90d385749abbb1b6f8711206
Length: 64

Hash Value 2:
a0badb200487fc4ce88b991c859ca928391ad8ed90d385749abbb1b6f8711206
Length: 64

Verification Result: Hashes Match

Modified Message:
Rayhan ParvaX

Modified Hash:
2861c0d0508720f9c78c663b51d257061fbdc9d7bb9be2062b59cd2eeb37b598
Length: 64

Avalanche Effect:
Changing one character completely changes the SHA-256 hash.

=====
PS C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab> |
```

Task 2(a): Task 2(a): Accepts ciphertext and recovers the plaintext

Source Code :

```
from collections import Counter

def count_letter_frequency(text):
    text = text.lower()
    text = ''.join(ch for ch in text if ch.isalpha())
    return Counter(text)

def decrypt_substitution(ciphertext, substitution_map):
    plaintext = ""

    for ch in ciphertext:
        if ch.lower() in substitution_map:
```

```

        plain = substitution_map[ch.lower()]
        plaintext += plain.upper() if ch.isupper() else plain
    else:
        plaintext += ch

    return plaintext

ciphertext = """
jgrmqoyghmvmvj wrwqfpw hgf fdqgfpfzr kbeebjizq qo cibzk.
lfafgqvzfzfw, eog wopf gfhwl phlr lolfdmfgqw blwbwq ol
kfwbylbly lfs fljgrmqbol wjvfpfw qvhq wffp qo qvfp qo
cf pogf wfjigf qvhl hlr oqvfg wjvfpf ol fhgqv.
qvf ileogqilhqf qgiqv vosfafg bw qvhq wijv wjvfpfw
hgf iwihzzr qgbabhq qo cgfhx.
"""

print("=" * 60)
print("      SUBSTITUTION CIPHER DECRYPTION")
print("=" * 60)

print("\nCiphertext:\n")
print(ciphertext)

print("\nLetter Frequency:\n")

frequency = count_letter_frequency(ciphertext)

for letter, count in sorted(frequency.items(), key=lambda x: x[1], reverse=True):
    print(f"{letter} : {count}")

substitution_map = {
    'a': 'v',
    'b': 'i',
    'c': 'b',
    'd': 'x',
    'e': 'f',
    'f': 'e',
    'g': 'r',
    'h': 'a',
    'i': 'u',
    'j': 'c',
    'k': 'd',
    'l': 'n',
    'm': 'p',
    'o': 'o',

```

```

        'p': 'm',
        'q': 't',
        'r': 'y',
        's': 'w',
        'v': 'h',
        'w': 's',
        'x': 'k',
        'y': 'g',
        'z': 'l'
    }

print("\nSubstitution Map (cipher -> plain):\n")

for c, p in sorted(substitution_map.items()):
    print(f"{c} -> {p}")

plaintext = decrypt_substitution(ciphertext, substitution_map)

print("\n" + "=" * 60)
print("Recovered Plaintext")
print("=" * 60)

print(plaintext)

print("\nProgram Finished Successfully.")

```

Output:

```

PS C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab> & C:\Users\hp\AppData\Local\Programs\Python\Python314\python.exe c:/Users/hp/OneDrive/Documents/Desktop/NSC_Lab/subs.py
=====
      SUBSTITUTION CIPHER DECRYPTION
=====

Ciphertext:

jgrmqoyghmvdj wrwqfpu hgf fdqsfprz kbeebjinz qo cibzk.
lfafgqvzfzfw, eog wopf gfhwoi phlr lolfdmfgow blwbq ol
kfwbylbly lfs fljgrmqbol wjvfpfw qvhq wffp qo qvfp qo
cf pogf wfjigf qvhi hlr oqvfg wjvfpf ol fhgqv.
qvf ileogqilhqf qgiqv vosfafg bw qvhq wjv wjvfpfw
hgf lwlhzr qgbabhq qo cgfhx.

Letter Frequency:

f : 37
q : 26
w : 21
g : 19
l : 17
o : 16
v : 15
h : 14
b : 12
p : 10
j : 9
i : 9
r : 7
z : 7
m : 4
e : 4
y : 3
k : 3
c : 3
a : 3
d : 2
s : 2
x : 1

Substitution Map (cipher -> plain):

a -> v
b -> i

```

```
s : z
x : l

Substitution Map (cipher -> plain):

a -> v
b -> i
c -> b
d -> x
e -> f
f -> e
g -> r
h -> a
i -> u
j -> c
k -> d
l -> n
m -> p
o -> o
p -> m
q -> t
r -> y
s -> w
v -> h
w -> s
x -> k
y -> g
z -> l

=====
Recovered Plaintext
=====

cryptographic systems are extremely difficult to build.
nevertheless, for some reason many nonexperts insist on
designing new encryption schemes that seem to them to
be more secure than any other scheme on earth.
the unfortunate truth however is that such schemes
are usually trivial to break.

Program Finished Successfully.
PS C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab>
```

Task 2(b): How Proma Found the Substitution Map

Proma started by counting how often each letter appears in the secret message she got. She used a Python program to do this.

The letters that appeared often were:

f, q, w, g, l o, v, h, b p

In the English language the letters that appear most often are E, T, A, O, I, N, S, H, R. So Proma used these letters as a starting point.

Iteration 1

The initial assumptions were done by Proma by aligning the most commonly occurring letters.

f → e
q → t

Substituting these characters enabled the recognition of some words.

Iteration 2

She then observed the repeated words and the English words that are common. In addition to that, she furthered the substitution list as follows:

w → s
g → r
l → n
o → o

With these substitutions, some more words became clear in the ciphertext.

Iteration 3

She continued to observe the repeated words and some common letter arrangements. The mappings used were as follows:

h → a
b → i
p → m
v → h

In this iteration, more words became readable.

Iteration 4

At last, she observed the structure of the sentence along with some partially recovered words. This helped her complete the substitution list as follows:

a → v
c → b
d → x
e → f
i → u
j → c
k → d
m → p
r → y
s → w
x → k
y → g
z → l

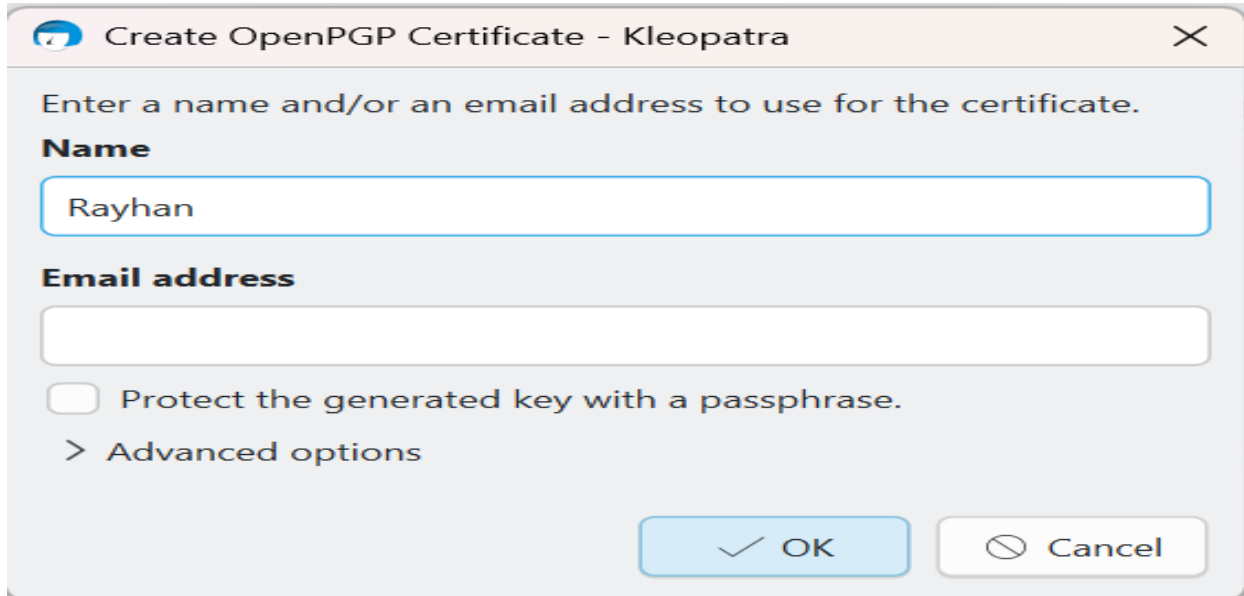
Now she could read most of the words correctly.

After trying times and looking at the frequencies of letters Proma finally figured out the secret code and found the original maintenance instructions.

This way of doing things is called frequency analysis, where you compare the frequencies of letters in the message with the frequencies of letters, in the English language until you get a message that makes sense. Proma used frequency analysis to solve the problem. Frequency analysis is a method that helps you decode messages by looking at the frequencies of letters.

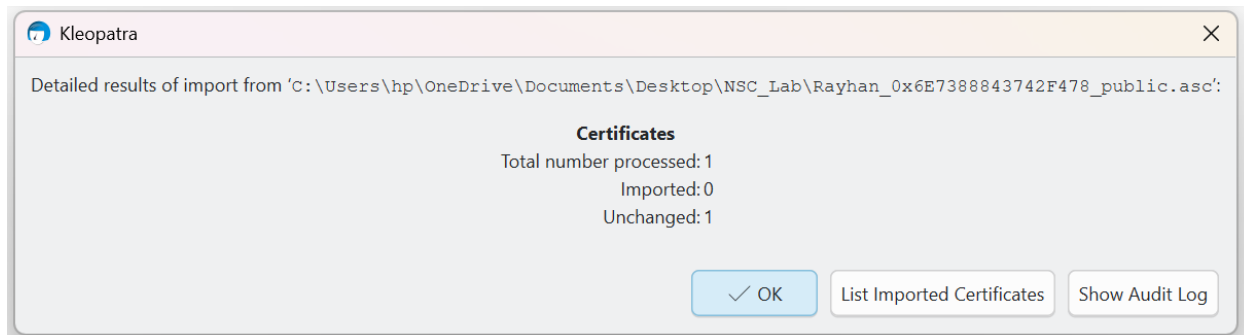
Task 3: Explain how you would utilize PGP tools based on Scenario III and complete all the associated tasks

Step 1: Create OpenPGP Key Pair



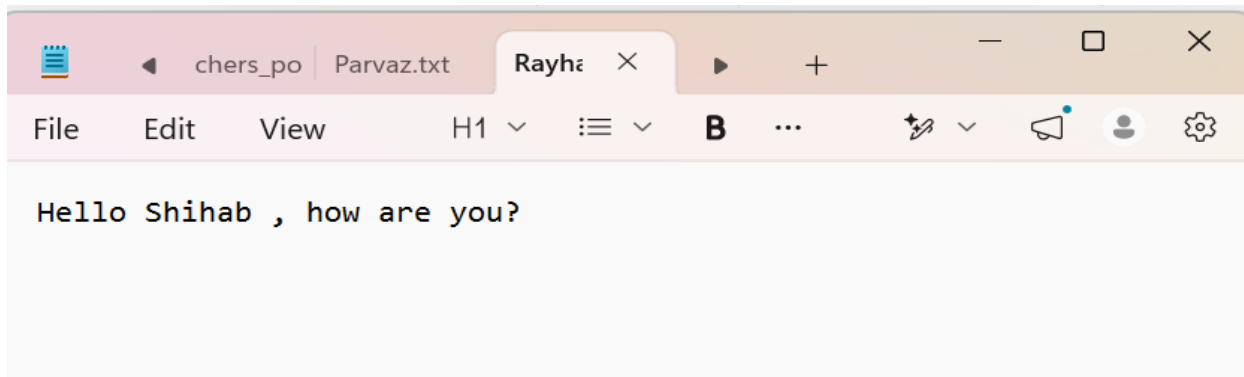
The screenshot shows the 'Create OpenPGP Certificate - Kleopatra' dialog box. It has a title bar with the Kleopatra logo and a close button. The main text says 'Enter a name and/or an email address to use for the certificate.' Below this, there are two input fields: 'Name' with the text 'Rayhan' and 'Email address' which is empty. There is a checkbox labeled 'Protect the generated key with a passphrase.' which is unchecked. Below the checkbox is a link '> Advanced options'. At the bottom right, there are two buttons: 'OK' with a checkmark icon and 'Cancel' with a circle-slash icon.

Step 2: Import Recipient's Public Key

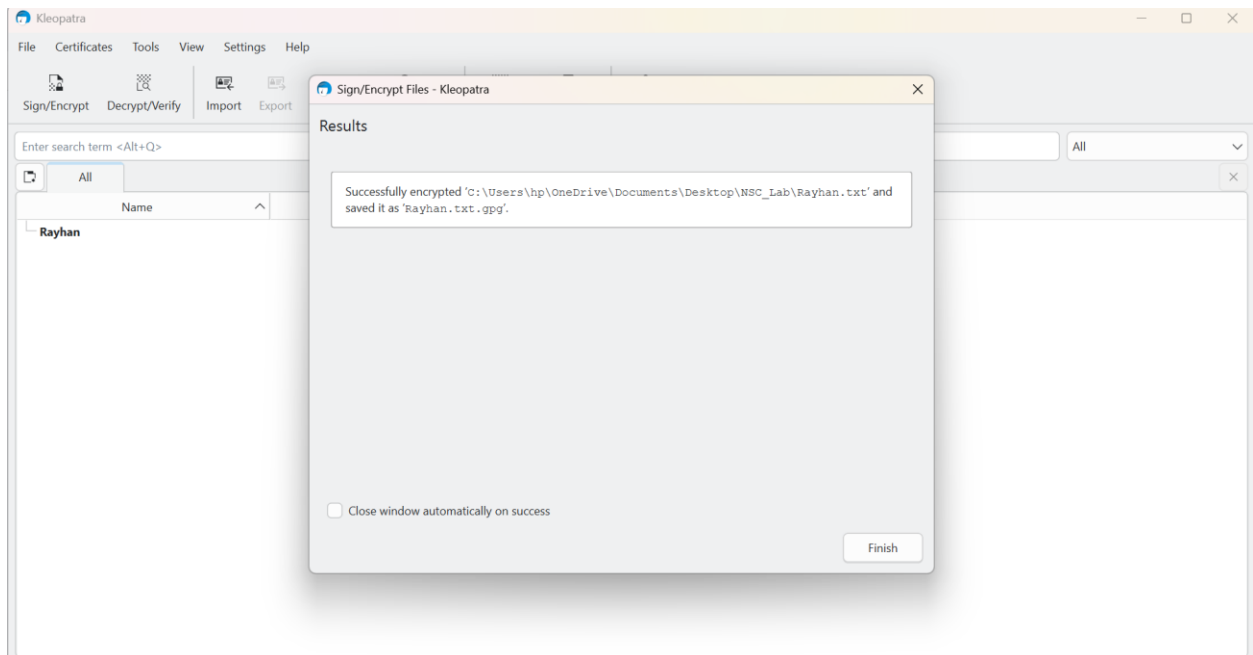


The screenshot shows the Kleopatra application window. The title bar says 'Kleopatra'. The main content area displays 'Detailed results of import from 'C:\Users\hp\OneDrive\Documents\Desktop\NSC_Lab\Rayhan_0x6E7388843742F478_public.asc':'. Below this, there is a section titled 'Certificates' with the following statistics: 'Total number processed: 1', 'Imported: 0', and 'Unchanged: 1'. At the bottom right, there are three buttons: 'OK' with a checkmark icon, 'List Imported Certificates', and 'Show Audit Log'.

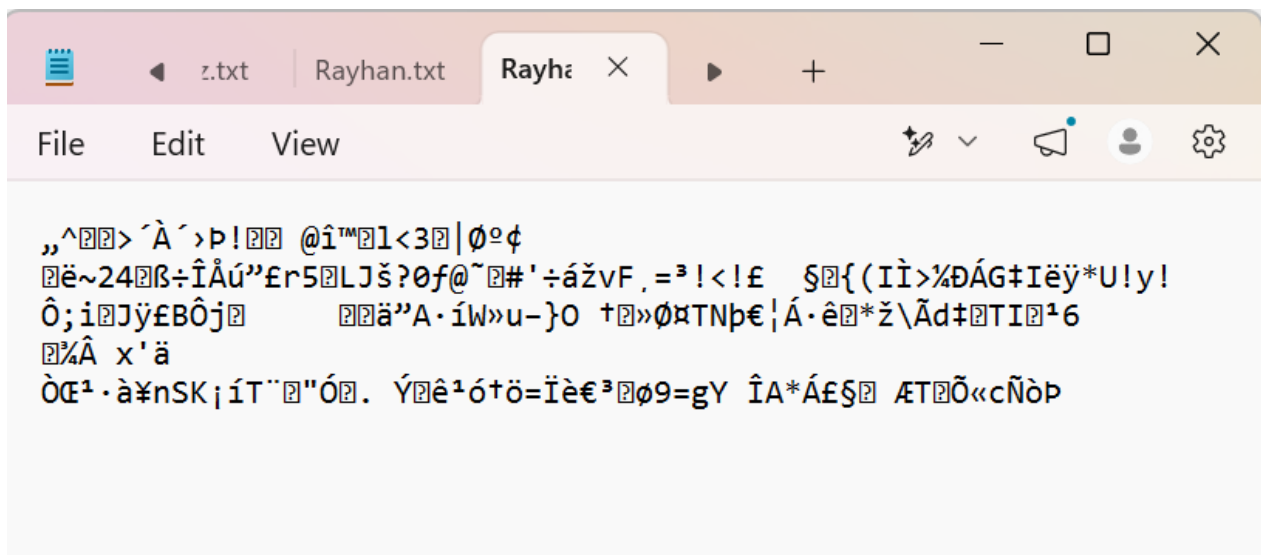
Step 3: Encrypt the Confidential File

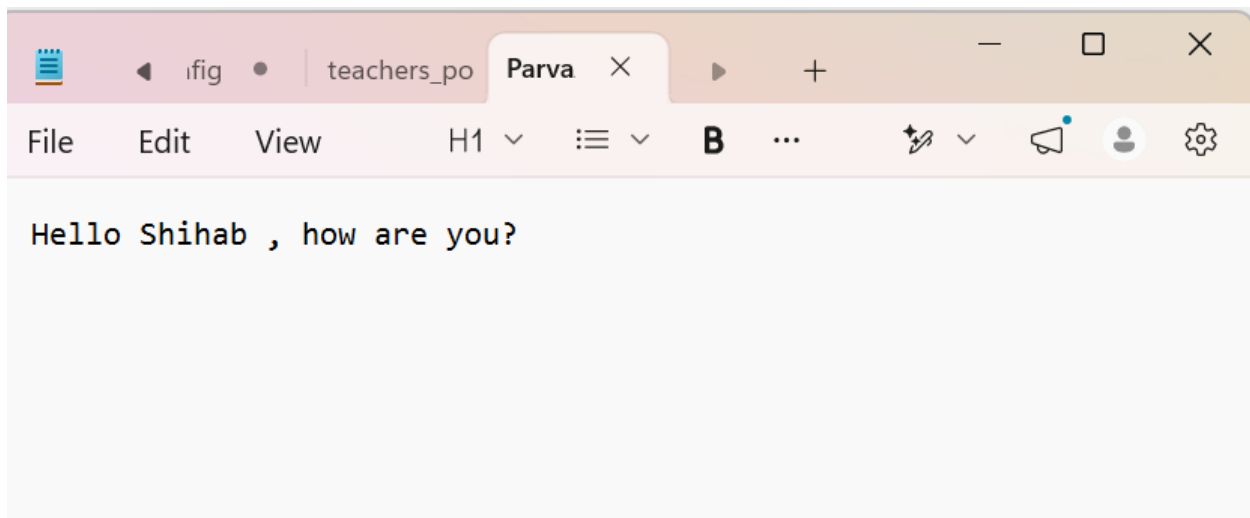
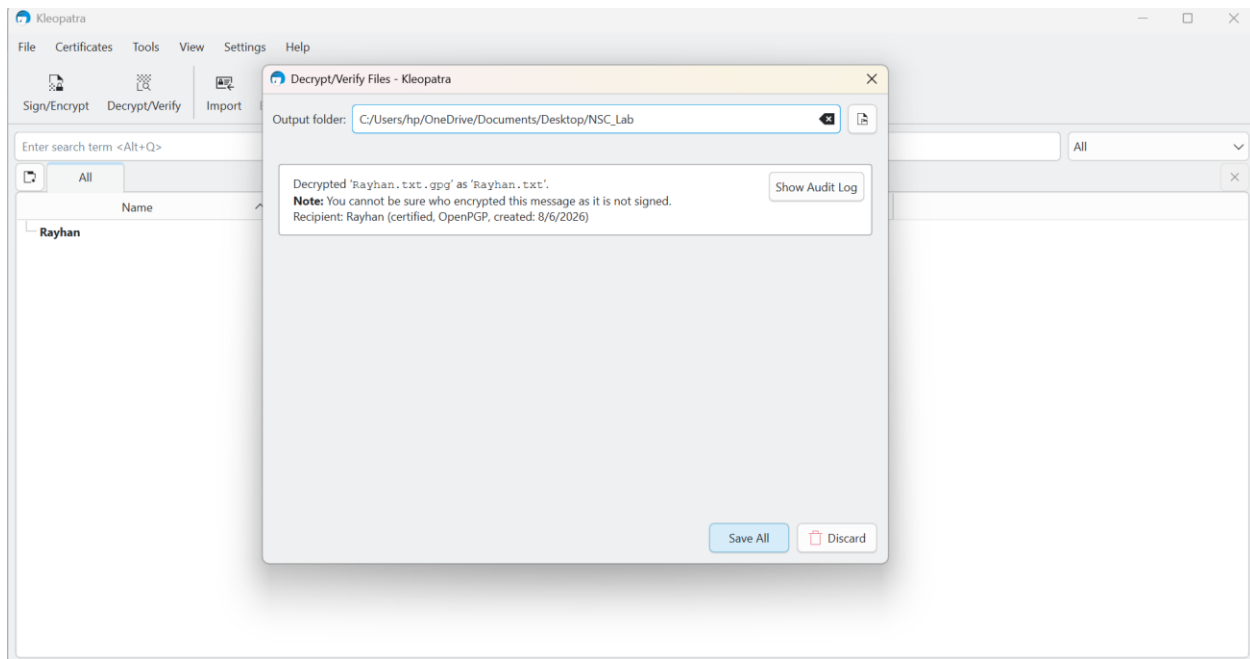


The screenshot shows a text editor window with a tab titled 'Rayhan'. The window has a menu bar with 'File', 'Edit', and 'View'. Below the menu bar, there are various icons for font settings (H1, bold, italic, underline, strikethrough, subscript, superscript, text color, background color, bulleted list, numbered list, indent, outdent, link, unlink, unlink all, unlink all selected), a search icon, a help icon, and a settings icon. The main text area contains the message 'Hello Shihab , how are you?'.



Step 4: Verify Encryption





After decrypting Rayhan.txt.gpg encrypted file with the private key in Kleopatra, the original file Rayhan.txt was retrieved with all the data intact. During the process of decryption, Kleopatra notified that the sender's identity was not authenticated because the file was not signed digitally. However, the decryption of the file was successful, which implies that the encryption process using PGP renders messages confidential. Therefore, the experiment performed has a positive result because it shows how to apply encryption in practice.

Prior to decrypting the file, it was necessary to set up the encryption process. In this case, the file was encrypted with the public key, which can only be decrypted with the private key. After decryption, the original file was retrieved successfully. The experiment can be considered positive because it shows that the PGP encryption process helps to maintain the confidentiality of the data since only people who have the private key can decrypt the file.